TALLER DEL 12-NOV · ARCHIVO POR ARCHIVO

Qué hace cada archivo del sitio

Dónde se creó, para qué sirve y qué hace cada parte del código



Programación IV — Frontend · UNIAJC · William Henao · 13 de noviembre de 2026

Mapa: dónde va cada archivo y para qué sirve

```
src/
├─ styles/variables.css        → los colores, tamaños y espacios (el sistema de diseño)
├─ styles/global.css           → reglas para todo el documento: body, títulos, enlaces
├─ layouts/Layout.astro        → el marco de TODAS las páginas: head, Header, Footer
├─ components/
│  ├─ layout/Header.astro      → marca y navegación con la página activa
│  ├─ layout/Footer.astro      → créditos, enlaces secundarios, crédito a la API
│  ├─ ui/Boton.astro           → botón o enlace con la misma apariencia
│  └─ ui/Etiqueta.astro        → píldora para categoría y tiempo
├─ pages/                      → cada archivo es una dirección del sitio
│  ├─ index.astro              → /              la portada
│  ├─ catalogo.astro           → /catalogo      (hoy la llenamos)
│  ├─ favoritos.astro          → /favoritos
│  ├─ contacto.astro           → /contacto
│  └─ acerca-de.astro          → /acerca-de
public/img/portada.svg       → archivos que se sirven tal cual
```

Cómo encajan: página → Layout → Header y Footer



- La página no repite el head, el Header ni el Footer: el Layout los pone.
- La página le pasa datos al Layout con props: titulo, descripcion, activo.
- El Layout le pasa activo al Header para marcar la página actual.
- Lo que la página escribe entre <Layout> y </Layout> cae en el <slot />.
- Boton y Etiqueta se usan dentro de las páginas, donde haga falta.

REVISIÓN

Errores que quedaron del live coding

Archivo	Escrito	Correcto
global.css	`body:{`	`body {`
variables.css	`--texto-M`	`--texto-m`
Layout	`initial-escale`	`initial-scale`
Layout	`content="{description"`	`content={descripcion}`
Layout	`"principal-ancho"`	`"principal--ancho"`
Header	`encabezado_interior`	`encabezado__interior`
Header	`strike-linecap`	`stroke-linecap`
Header	`import … "astro:schema"`	`(se borra)`

Solo el primero rompía el build.

Los demás no dan error: el sitio "funciona", pero algo no se aplica. El encabezado no pasaba a fila en escritorio por un guion bajo de menos.

HTML y CSS no avisan: un nombre mal escrito simplemente no se aplica.

variables.css y global.css: los estilos de todo el sitio

```css
/* variables.css */
:root {
  --color-marca: #b4451f;
  --texto-m: 1.125rem;
  --esp-4: 1.5rem;
  --radio-m: 14px;
}

/* global.css */
body {
  font-family: var(--fuente-texto);
  background: var(--color-lienzo);
}
.contenedor {
  max-width: var(--ancho-maximo);
  margin: 0 auto;
}
```

Sirve para: guardar las decisiones de diseño en un solo lugar. Ningún componente escribe un color o un tamaño suelto.

1. **:root**
   Declara las variables para toda la página. Se nombran con dos guiones: --color-marca.

2. **var(--nombre)**
   Las usa. Si cambio --color-marca, cambia en todo el sitio.

3. **global.css**
   Reglas que aplican a todo: body, títulos, enlaces, foco y la clase .contenedor.

4. **¿Quién los carga?**
   El Layout, con import. Por eso toda página que use el Layout tiene los estilos.

Layout.astro: el marco de todas las páginas

```
---
import Header from "../components/layout/Header.astro";
import Footer from "../components/layout/Footer.astro";
import "../styles/variables.css";
import "../styles/global.css";
const { titulo, descripcion, activo,
        ancho = "normal" } = Astro.props;
---
<html lang="es">
  <head>
    <title>{titulo}</title>
  </head>
  <body>
    <a class="saltar" href="#contenido">Saltar…</a>
    <Header activo={activo} />
    <main id="contenido">
      <slot />
    </main>
    <Footer />
  </body>
</html>
```

Sirve para: escribir una sola vez lo que se repite en todas las páginas: el <head>, el Header, el Footer y los estilos.

1. **import**
   Trae los componentes y los dos CSS. Así todas las páginas tienen estilos.

2. **Astro.props**
   Recibe los datos que manda cada página. ancho = "normal" es el valor si no lo mandan.

3. **<slot />**
   El hueco donde entra el contenido de cada página.

4. **Saltar al contenido**
   Primer enlace para quien navega con teclado: se salta el menú.

Header.astro: la navegación con la página activa

```
---
const base = import.meta.env.BASE_URL;
const { activo } = Astro.props;
const enlaces = [
  { nombre: "Inicio",   ruta: "",         clave: "inicio" },
  { nombre: "Catalogo", ruta: "catalogo", clave: "catalogo" },
  …
];
---
<nav aria-label="Navegacion principal">
  <ul>
    {enlaces.map((e) => (
      <li><a href={`${base}${e.ruta}`}
        aria-current={activo === e.clave
          ? "page" : undefined}>
        {e.nombre}</a></li>
    ))}
  </ul>
</nav>
```

Sirve para: mostrar la marca y los enlaces principales, y marcar en cuál página está el usuario.

1. **Arreglo enlaces**
   Los enlaces son datos. Para agregar uno, se agrega una línea, no HTML.

2. **.map()**
   Recorre el arreglo y dibuja un <li> por cada enlace.

3. **aria-current="page"**
   Marca el enlace de la página actual. El lector de pantalla dice "página actual" y el CSS lo pinta.

4. **${base}**
   Agrega la carpeta del repositorio. Sin él, los enlaces dan 404 en GitHub Pages.

Footer.astro: el pie de página

```
---
const base = import.meta.env.BASE_URL;
const anio = new Date().getFullYear();
---
<footer class="pie">
  <div class="contenedor pie__interior">
    <div class="pie__bloque">  … marca …  </div>
    <nav class="pie__bloque" aria-label="Enlaces del pie">
      <li><a href={`${base}catalogo`}>Catalogo</a></li>
    </nav>
    <div class="pie__bloque">
      Las recetas provienen de <a href="…">TheMealDB</a>
    </div>
  </div>
  <div class="contenedor pie__legal">
    <p>Proyecto academico … · {anio}</p>
  </div>
</footer>
```

Sirve para: dar el crédito a la API, enlaces para quien llegó al final y el aviso legal. Aparece en todas las páginas porque lo pone el Layout.

1. **Tres bloques**
   Marca, enlaces y datos. En el celular van uno debajo del otro; en escritorio, en tres columnas.

2. **{anio}**
   JavaScript en el frontmatter: el año se calcula solo al construir el sitio.

3. **Crédito a la API**
   Obligatorio: usar datos de otro sin nombrarlo es una falta.

4. **aria-label**
   Hay dos <nav> en la página; la etiqueta los distingue para el lector de pantalla.

Footer.astro: de una a tres columnas con Grid

```css
.pie__interior {
  display: grid;
  gap: var(--esp-5);
}

@media (min-width: 768px) {
  .pie__interior {
    grid-template-columns: 1.6fr 1fr 1.2fr;
  }
}

.pie__lista a:hover {
  color: var(--color-marca);
}
```

Sirve para: acomodar los tres bloques: apilados en el celular y en tres columnas desde 768 px, sin escribir nada especial para el celular.

1. **display: grid**
   Sin columnas definidas, Grid es una sola columna: los bloques se apilan.

2. **@media (min-width)**
   Móvil primero: lo de afuera es el celular; adentro, lo que cambia en pantallas grandes.

3. **fr**
   Fracción del espacio libre. 1.6fr es un poco más ancho que 1fr.

4. **gap**
   El espacio entre bloques. Reemplaza los márgenes entre hijos.

Boton.astro: un enlace o un botón con la misma apariencia

```
---
const {
  href,
  variante = "primario",
  tamano = "normal",
  tipo = "button",
  ...resto
} = Astro.props;

const Etiqueta = href ? "a" : "button";
const clases = ["boton", `boton--${variante}`,
                `boton--${tamano}`].join(" ");
---
<Etiqueta class={clases} href={href}
  type={href ? undefined : tipo} {...resto}>
  <slot />
</Etiqueta>
```

Sirve para: tener un solo botón para todo el sitio. Decide solo si dibuja un <a> o un <button>.

1. **Props con valor por defecto**
   Si no mandan variante, es "primario"; si no mandan tamano, es "normal".

2. **Etiqueta = href ? "a" : "button"**
   Con href es un enlace (lleva a otra página). Sin href es un botón (hace algo aquí).

3. **clases**
   Arma "boton boton--primario boton--normal" para que el CSS lo pinte.

4. **...resto y <slot />**
   Cualquier otro atributo pasa directo; el texto del botón entra por el slot.

Boton.astro: tres variantes y 44 px de alto

```
.boton {
  display: inline-flex;
  align-items: center;
  min-height: 44px;
  border-radius: var(--radio-s);
}
.boton--primario {
  background: var(--color-marca);
  color: #ffffff;
}
.boton--secundario {
  border-color: var(--color-marca-borde);
}
.boton--fantasma {
  background: transparent;
}
.boton:active { transform: translateY(1px); }
```

Sirve para: que todos los botones se vean iguales y se puedan tocar con el dedo.

1. **min-height: 44px**
   El tamaño que un dedo acierta en el celular. Con menos, se toca el botón de al lado.

2. **boton--variante**
   Primario para la acción principal (una por pantalla), secundario y fantasma para el resto.

3. **inline-flex**
   Centra el texto (y un ícono si lo hay) dentro del botón.

4. **:hover y :active**
   Respuesta visual al pasar el mouse y al hacer clic.

Etiqueta.astro: la píldora de categoría y tiempo

```
---
const { tono = "neutro" } = Astro.props;
---

<span class={`etiqueta etiqueta--${tono}`}>
  <slot />
</span>

<style>
  .etiqueta {
    padding: 0.4rem 0.7rem;
    border-radius: var(--radio-pildora);
    white-space: nowrap;
  }
  .etiqueta--marca {
    background: var(--color-marca-suave);
  }
</style>
```

Sirve para: mostrar datos cortos (categoría, tiempo, dificultad) como píldoras. Hoy la usamos dentro de las tarjetas.

1. **tono**
   Prop con dos valores: "neutro" (por defecto) y "marca" (resaltada).

2. **Template literal**
   `etiqueta etiqueta--${tono}` arma la clase con el valor de la prop.

3. **radio-pildora (999px)**
   Bordes totalmente redondos, sin importar el tamaño.

4. **white-space: nowrap**
   "45 min aprox." nunca se parte en dos líneas.

index.astro: la portada usa el Layout y el Boton

```
---
import Layout from "../layouts/Layout.astro";
import Boton from "../components/ui/Boton.astro";
const base = import.meta.env.BASE_URL;
---
<Layout titulo="SaborU — Recetas…"
  descripcion="…" activo="inicio" ancho="ancho">
  <section class="portada">
    <h1>Decide que cocinar esta noche…</h1>
    <div class="portada__acciones">
      <Boton href={`${base}catalogo`} tamano="grande">
        Ver el catalogo</Boton>
    </div>
    <img src={`${base}img/portada.svg`} alt="…"
         width="640" height="480" />
  </section>
  <section> … destacadas … </section>
  <section> … como funciona … </section>
</Layout>
```

Sirve para: ser la página de inicio (/). Presenta el producto y lleva al catálogo.

1. **Envuelta en <Layout>**
   No tiene <html> ni <head>: los pone el Layout. activo="inicio" marca Inicio en el Header.

2. **ancho="ancho"**
   Quita el espacio de arriba para que el fondo de la portada llegue hasta el Header.

3. **Tres <section>**
   Portada, destacadas y cómo funciona: un bloque del mockup = una sección.

4. **width y height**
   Reservan el espacio de la imagen antes de que cargue: la página no salta.

index.astro: cada bloque del mockup, un bloque del CSS

```css
.portada__interior {
  display: grid;
  gap: var(--esp-5);
}
@media (min-width: 900px) {
  .portada__interior {
    grid-template-columns: 1.05fr 0.95fr;
  }
}
.portada__acciones {
  display: flex;
  flex-wrap: wrap;
  gap: var(--esp-2);
}
@media (min-width: 768px) {
  .pasos { grid-template-columns: repeat(3, 1fr); }
}
```

Sirve para: acomodar la portada: una columna en el celular y dos en escritorio, como en el mockup.

1. **Texto | imagen**
   Grid de 2 columnas desde 900 px. Antes, una debajo de la otra.

2. **Dos botones**
   Flexbox con flex-wrap: si no caben, el segundo baja.

3. **Tres pasos**
   Grid de 3 columnas iguales desde 768 px.

4. **¿Grid o Flexbox?**
   Grid cuando hay columnas; Flexbox cuando es una fila de cosas que fluyen.

catalogo, favoritos y contacto: la misma plantilla

```
---
import Layout from "../layouts/Layout.astro";
---
<Layout
  titulo="Catalogo — SaborU"
  descripcion="Catalogo de recetas…"
  activo="catalogo"
>
  <section class="contenedor">
    <p class="rotulo">Catalogo</p>
    <h1>Que puedes cocinar hoy</h1>
    <p class="entrada">Filtra por categoria…</p>
    <div class="pendiente">
      Se construye en la proxima clase.
    </div>
  </section>
</Layout>
```

Sirve para: que todas las direcciones del Header existan desde ya (ningún enlace da 404), aunque su contenido llegue después.

1. **El nombre del archivo es la dirección**
   catalogo.astro → /catalogo. favoritos.astro → /favoritos.

2. **Solo cambian tres cosas**
   titulo, descripcion y activo (y los textos). La estructura es la misma.

3. **activo = clave del Header**
   Si no coincide exactamente, ningún enlace queda marcado.

4. **.pendiente**
   Un recuadro punteado que avisa que esa parte se construye después.

acerca-de.astro: la única página terminada

```
<div class="disposicion">
  <article class="texto"> … problema, personas … </article>
  <aside class="ficha">
    <dl class="ficha__lista">
      <div><dt>Generador</dt><dd>Astro</dd></div>
      <div><dt>Datos</dt><dd>TheMealDB</dd></div>
    </dl>
  </aside>
</div>

@media (min-width: 900px) {
  .disposicion { grid-template-columns: 1.3fr 0.7fr; }
  .ficha { position: sticky; top: 7.5rem; }
}
```

Sirve para: explicar el problema, para quién es el producto, las decisiones de diseño y los créditos. Es texto fijo, por eso ya queda lista.

1. **<article> y <aside>**
   El texto principal y la información complementaria. Etiquetas con significado, no solo .

2. **<dl>, <dt>, <dd>**
   Lista de pares nombre–valor: "Datos: TheMealDB".

3. **Grid 1.3fr 0.7fr**
   Desde 900 px: texto a la izquierda, ficha a la derecha.

4. **position: sticky**
   La ficha se queda quieta al hacer scroll, debajo del Header (top: 7.5rem).

PARA RESOLVER EN CLASE

Preguntas que les pueden surgir

¿Qué es una prop?

Un dato que un componente le pasa a otro, como un parámetro: <Layout titulo="Inicio">.

¿Qué hace <slot />?

Es el hueco donde entra lo que se escribe entre la etiqueta de apertura y la de cierre.

¿Por qué ${base} en cada enlace?

GitHub Pages publica dentro de una carpeta. Sin base, el enlace da 404.

¿Qué es lo de arriba entre ---?

El frontmatter: JavaScript que corre al construir el sitio, no en el navegador.

¿Por qué mi estilo no se aplica?

Un nombre mal escrito: clase, variable o guion. CSS no da error, solo lo ignora.

¿Grid o Flexbox?

Grid si hay columnas (filas y columnas a la vez); Flexbox si es una fila que fluye.

Revisión rápida de cada grupo

1. ¿La URL publicada abre y se parece al mockup?
2. ¿Cambia la página activa al navegar?
3. ¿Algún enlace del Header o del Footer da 404?
4. ¿A 320 px la consola responde true?
5. ¿Aparecen los dos integrantes en Contributors?

```
// en la consola, a 320 px
document
  .documentElement
  .scrollWidth ===
document
  .documentElement
  .clientWidth
```

Hoy todos arrancamos desde main: git pull antes de empezar.